

M forecasting, Probability consistency, Token-constrained decoding, Adversarial rewriting, Dutch Book, Decoding-time alignment

HybridACD: Hybrid Adversarial Constrained Decoding for Logically Consistent Large Language Model Forecasting

Kiet Tran Anh^{1*}

^{1*}Faculty of Information Science and Engineering, University of
Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam.

Corresponding author(s). E-mail(s): 23520820@gm.uit.edu.vn;

Abstract

This report investigates the problem of *probability consistency* in large language models (LLMs) when they are used as forecasting agents. Although LLMs can produce plausible probability estimates for individual questions, their predictions may still violate basic logical relations such as negation, paraphrase, conjunction, disjunction, and conditional probability. Such violations reduce the reliability of forecasting systems and can be interpreted as Dutch Book vulnerabilities in probability space.

To address this problem, we propose **HybridACD** (*Hybrid Adversarial Consistency Decoding*), a decoding-time intervention architecture that combines two mechanisms: (i) adversarial rewriting, which generates question variants that are semantically equivalent but difficult to solve through shallow surface-pattern matching, and (ii) token-constrained decoding, which restricts the output probability to a valid interval derived from consistency rules. Unlike costly post-hoc calibration or model fine-tuning that may suffer from Goodhart effects, HybridACD keeps the model weights unchanged and enforces constraints only at the final probability extraction step.

Experiments on six LLMs show that HybridACD reduces the average Aggregated Violation Score from **0.1522** to **0.0053** (**−95.5%**), while improving the Brier Score by **2.4%** to **17.3%** on the models with full evaluation. These results suggest that decoding-time constraints are an effective direction for improving the probability consistency of LLM forecasters without degrading forecasting accuracy.

Contents

1	Introduction	3
2	Related Work	4
2.1	Probability Consistency in LLM Forecasters	4
2.2	Preference Optimization and Self-Consistency	4
2.3	Constrained Decoding and Decoding-Time Alignment	5
2.4	Controlled Prompting and Adversarial Robustness	5
3	Method	5
3.1	Problem Formulation	5
3.2	Consistency Rules	6
3.3	Overview of the HybridACD Architecture	6
3.4	Computing Probability Bounds	7
3.5	Token-Constrained Decoding	7
4	Experimental Results	8
4.1	Experimental Setup	8
4.2	Reduction in Consistency Violations	8
4.3	Forecasting Accuracy	9
4.4	Comparison with Baselines	9
4.5	Ablation Study	10
5	Discussion	11
6	Conclusion	11

1 Introduction

Large language models (LLMs) are increasingly used in probabilistic forecasting tasks, where a model must assign a value $p \in [0, 1]$ to the likelihood of a future event. Unlike standard classification tasks, forecasting requires the model not only to provide a plausible answer for each individual question, but also to maintain a coherent belief system across multiple logically related questions. For example, if a model predicts $P(A) = 0.70$, then the probability of the negated event $\neg A$ should be close to 0.30; otherwise, the system violates the additivity axiom of probability.

Recent studies on LLM forecasters show that consistency scores can serve as early evaluation signals when ground truth has not yet become available [1]. This is especially important in future-event forecasting, where waiting for event resolution may take months or years. However, good calibration on individual questions does not guarantee consistency over groups of logically related questions. An LLM may obtain a low Brier Score while still producing contradictory probabilities for negated events, equivalent events, or compound events.

This issue can be formalized through the Dutch Book argument: if a set of probability forecasts is internally inconsistent, an adversary can construct a portfolio of

bets that guarantees positive profit regardless of the actual outcome. Therefore, probability consistency is not merely a formal requirement, but also a necessary condition for an LLM forecaster to be considered a reliable decision-making system. Post-hoc methods such as arbitrage calibration can reduce violations, but they often require many model calls and may incur high cost. In contrast, directly optimizing model weights for consistency may lead to a Goodhart effect: the model learns to optimize the consistency metric through neutral predictions such as $P = 0.5$, thereby reducing its practical forecasting value.

This report focuses on **HybridACD**, a hybrid architecture that combines adversarial rewriting and token-constrained decoding. The central idea is to impose probability constraints at decoding time, before the final probability is committed. This approach has three advantages: it does not require model retraining, it avoids iterative post-hoc calibration, and it can directly convert probability rules into valid intervals for output tokens.

The main contributions of this report are:

1. We present the HybridACD framework for LLM forecasting, combining adversarial rewriting with decoding-time probability constraints.
2. We formalize how probability consistency rules can be converted into bounds $[\ell, u]$ for Token-Constrained Decoding (TCD).
3. We empirically evaluate HybridACD across multiple LLMs, focusing on three criteria: reducing consistency violations, improving Brier Score, and controlling inference cost.

2 Related Work

2.1 Probability Consistency in LLM Forecasters

Paleka et al. [1] propose evaluating LLM forecasters through consistency checks across multiple logically related questions. Instead of scoring forecasts only after events are resolved, this approach tests whether the probabilities generated by a model satisfy relations such as negation, implication, conjunction, disjunction, and conditional probability. This work provides the direct foundation for HybridACD, because the goal of the system is not only to forecast each question accurately, but also to maintain a valid probability distribution over the entire question set.

At the mechanistic level, LLMs generate text by predicting the next token from a conditional probability distribution [2]. However, internal token probabilities do not always correspond to the logical validity of the final answer. Analyses of probability consistency show that models may prefer token sequences that are natural in language but inconsistent in probability space [3, 4]. This explains why a separate intervention at the numerical output layer is needed, instead of relying only on prompting or sequence probability.

2.2 Preference Optimization and Self-Consistency

One direction for improving reasoning is to use internal probability signals during optimization. Yang et al. [5] propose Probability-Consistent Preference Optimization

(PCPO), where preference selection is based not only on answer correctness but also on token-level probability consistency. Zhou et al. [6] connect internal probability with self-consistency by using perplexity to select or prune unreliable reasoning paths. These studies show that internal probability can support reasoning, but most of them act during training or sample selection. HybridACD differs by imposing constraints directly at the moment when the final probability is decoded.

2.3 Constrained Decoding and Decoding-Time Alignment

Constrained decoding refers to methods that control LLM outputs by restricting valid tokens during generation. Koo et al. [7] use automata to represent formal constraints, which is especially useful for structured outputs such as JSON, API calls, or expressions with a predefined grammar. Yao et al. [8] show that Token Constraint Decoding can improve robustness in question answering when the output must belong to a valid token set. Huang et al. [9] extend this idea through DeAL, a decoding-time alignment framework that can combine multiple rewards and declarative constraints.

However, constrained decoding also involves trade-offs. Schall and de Melo [10] show that structural constraints can reduce generation performance, especially for instruction-tuned models, because they may push the model away from the natural language patterns it prefers. Therefore, HybridACD applies constraints only during the numerical probability extraction step rather than constraining the entire reasoning process. This design preserves the model’s free-form reasoning ability while controlling the final probability value so that it satisfies probability axioms.

2.4 Controlled Prompting and Adversarial Robustness

Mousavi and Termehchy [11] show that controlled prompting and decoding can reduce inconsistent answers across equivalent queries. In HybridACD, adversarial rewriting is used for a similar goal but in the context of probabilistic forecasting: questions are transformed syntactically through entity substitution, negation lifting, or nested conditions while preserving the same resolution criteria. This helps prevent the model from merely learning the surface patterns of consistency checks.

Taken together, these research directions reveal a gap at the intersection of three elements: logical consistency evaluation, internal probability signals, and decoding-time output control. HybridACD is designed to address this gap through a compact, training-free pipeline applicable to many base LLMs.

3 Method

3.1 Problem Formulation

Let $\mathcal{Q}$ denote the set of binary forecasting questions. A forecaster $\mathcal{F}$ maps each question $Q \in \mathcal{Q}$ to a subjective probability:

$$\mathcal{F} : \mathcal{Q} \rightarrow [0, 1], \quad p = \mathcal{F}(Q). \quad (1)$$

A question set $T = \{Q_1, \dots, Q_k\}$ is called a consistency set if the questions are connected by a logical relation $\mathcal{L}$ that induces a probability constraint $\mathcal{C}(p_1, \dots, p_k)$. For example, under the negation rule, $\mathcal{C}$ requires $p_P + p_{\neg P} = 1$.

The goal of HybridACD is to generate predictions $\hat{p}_1, \dots, \hat{p}_k$ such that:

$$\mathcal{C}(\hat{p}_1, \dots, \hat{p}_k) = 0 \quad \text{and} \quad \mathbb{E}[(\hat{p} - y)^2] \text{ is minimized as much as possible,} \quad (2)$$

where $(\hat{p} - y)^2$ is the Brier Score and $y \in \{0, 1\}$ is the realized outcome.

3.2 Consistency Rules

HybridACD uses ten probability consistency rules, summarized in Table 1. Each rule is converted into a feasible interval $[\ell, u]$ for the current prediction based on previously obtained predictions.

Table 1 Ten probability consistency rules and their mathematical constraints. For each rule, ℓ and u denote the lower and upper bounds imposed on the target variable given previously known values.

#	Rule	Constraint $\mathcal{C}(p_1, \dots, p_k)$	Set Key
1	NEGATION	$p_P + p_{\neg P} = 1$	$\{P, \neg P\}$
2	PARAPHRASE	$p_P = p_{P'}$	$\{P, P'\}$
3	CONSEQUENCE	$P \models B \Rightarrow p_P \leq p_B$	$\{P, B\}$
4	AND	$\max(0, p + q - 1) \leq p_{P \wedge Q} \leq \min(p, q)$	$\{P, Q, P \wedge Q\}$
5	OR	$\max(p, q) \leq p_{P \vee Q} \leq \min(1, p + q)$	$\{P, Q, P \vee Q\}$
6	BUT	$p_{P \vee Q} = p_P + p_{Q \wedge \neg P}$	$\{P, Q \wedge \neg P, P \vee Q\}$
7	CONDITIONAL	$p_{P \wedge Q} = p_P \cdot p_{Q P}$	$\{P, Q P, P \wedge Q\}$
8	CONDCOND	$p_{P \wedge Q \wedge R} = p_P \cdot p_{Q P} \cdot p_{R P \wedge Q}$	$\{P, Q P, R P \wedge Q, P \wedge Q \wedge R\}$
9	AND-OR	$p_P + p_Q = p_{P \vee Q} + p_{P \wedge Q}$	$\{P, Q, P \wedge Q, P \vee Q\}$
10	EXPECTED EVIDENCE	$p_P = p_Q p_{P Q} + (1 - p_Q) p_{P \neg Q}$	$\{P, Q, P Q, P \neg Q\}$

3.3 Overview of the HybridACD Architecture

HybridACD consists of two main components. The first component is the *Adversarial Input Agent*, which uses an auxiliary language model to rewrite a forecasting question so that it becomes syntactically more complex while preserving the same resolution criteria. The second component is *Token-Constrained Decoding*, which computes the valid probability interval and forces the parsing model to return a value inside that interval.

For a question Q_i , the adversarial agent generates the variant:

$$Q'_i = \mathcal{M}_{adv}(Q_i; \tau = 0.7), \quad \text{Sem}(Q'_i) = \text{Sem}(Q_i). \quad (3)$$

The transformations include subordinate-clause insertion, entity substitution, negation lifting, and nested conditionals. The goal is not to change the label, but to reduce the model's ability to rely on shallow pattern matching.

The main model then generates free-form reasoning:

$$\text{cot}_i = \mathcal{M}(Q'_i). \quad (4)$$

This reasoning step is not constrained, allowing the model to retain the benefits of Chain-of-Thought reasoning. Constraints are applied only when extracting the final probability.

3.4 Computing Probability Bounds

Assume that previous predictions are $\hat{p}_1, \dots, \hat{p}_{i-1}$. Given the logical rule $\mathcal{L}$, HybridACD computes:

$$\ell_i = \max(0, \phi_\ell(\hat{p}_1, \dots, \hat{p}_{i-1}; \mathcal{L})), \quad u_i = \min(1, \phi_u(\hat{p}_1, \dots, \hat{p}_{i-1}; \mathcal{L})). \quad (5)$$

For example, under the COND rule, if $\hat{p}_P$ and $\hat{p}_{Q|P}$ are known, then the valid value for $P \wedge Q$ is:

$$\ell = u = \hat{p}_P \cdot \hat{p}_{Q|P}. \quad (6)$$

Under the AND rule, the valid value for $P \wedge Q$ lies in the interval:

$$\max(0, \hat{p}_P + \hat{p}_Q - 1) \leq \hat{p}_{P \wedge Q} \leq \min(\hat{p}_P, \hat{p}_Q). \quad (7)$$

3.5 Token-Constrained Decoding

TCD operates on the discrete probability value set:

$$\mathcal{V}_{num} = \{0.00, 0.01, \dots, 1.00\}.$$

Each value $v \in \mathcal{V}_{num}$ is tokenized to identify the token representing the probability. Given a valid interval $[\ell_i, u_i]$, HybridACD constructs the following logit bias:

$$b_{t_v} = \begin{cases} 0, & \text{if } v \in [\ell_i, u_i], \\ -100, & \text{if } v \notin [\ell_i, u_i]. \end{cases} \quad (8)$$

Probability tokens outside the valid interval are strongly penalized, while valid tokens remain unchanged. If the API does not support logit bias or the parsing process fails, the system uses a deterministic fallback:

$$\hat{p}_i = \text{clip}(p_{raw}, \ell_i, u_i). \quad (9)$$

Therefore, the final output always lies within the probability interval derived from the consistency rule.

Algorithm 1 HybridACD Inference Pipeline

Require: Consistency set $T = \{(Q_1, \text{key}_1), \dots, (Q_k, \text{key}_k)\}$, rule $\mathcal{L}$

Ensure: Predictions $\{\hat{p}_1, \dots, \hat{p}_k\}$ satisfying $\mathcal{C}(p_1, \dots, p_k)$

```
1:  $\mathcal{H} \leftarrow \{\}$  ▷ History of previous predictions
2: for  $i = 1, \dots, k$  do
3:    $Q'_i \leftarrow \mathcal{M}_{\text{adv}}(Q_i; \tau = 0.7)$  ▷ Adversarial rewriting
4:    $[\ell_i, u_i] \leftarrow \phi(\mathcal{H}, \text{key}_i, \mathcal{L})$  ▷ Bound computation
5:    $\text{cot}_i \leftarrow \mathcal{M}(Q'_i)$  ▷ Chain-of-Thought generation
6:    $B \leftarrow \{t_v \mapsto b_{t_v}\}_{v \in \mathcal{V}_{\text{num}}}$  ▷ Logit-bias map construction
7:    $\hat{p}_i \leftarrow \text{parse}(\text{cot}_i \mid B)$  ▷ TCD-constrained extraction
8:    $\hat{p}_i \leftarrow \text{clip}(\hat{p}_i, \ell_i, u_i)$  ▷ Deterministic fallback
9:    $\mathcal{H}[\text{key}_i] \leftarrow \hat{p}_i$  ▷ Store for later bounds
10: end for
11: return  $\{\hat{p}_1, \dots, \hat{p}_k\}$ 
```

4 Experimental Results

4.1 Experimental Setup

HybridACD is evaluated on a forecasting benchmark composed of question sets with logical relations. The tested models include Gemini-2.5-Flash, GPT-4o-mini, GPT-5.4-mini, MiniMax-M3, Mistral-Medium-3.5-128B, and Mistral-Small-4-119B. For each model, this report compares the BasicForecaster baseline with the full HybridACD variant.

The main metrics are: (i) Aggregated Violation Score (AVS, lower is better), which measures consistency violations through an arbitrage-based metric; (ii) the Frequentist score, which measures deviation from the expected distribution; (iii) Brier Score, which measures real forecasting error; and (iv) Calibration Error, which measures the deviation between predicted confidence and empirical accuracy.

4.2 Reduction in Consistency Violations

Table 2 shows that HybridACD reduces AVS across all six models. The average reduction reaches 95.5%, from 0.1522 to 0.0053. This indicates that decoding-time constraints can effectively control logical violations without being strongly dependent on the base model architecture.

The largest improvements appear for MiniMax-M3 (−99.6%) and GPT-5.4-mini (−97.8%). GPT-5.4-mini has a substantially higher baseline AVS than the other models, but after HybridACD its AVS decreases to 0.0107. This result suggests that TCD is especially useful when a model tends to produce extreme or unstable probabilities across logically related questions.

Table 2 Aggregated Violation Score (AVS, ↓) and Frequentist score (↓) for BasicForecaster versus HybridACD on the Scraped benchmark. Best values are shown in bold.

Model	AVS (default)		Frequentist		Improvement	
	Baseline	HybridACD	Baseline	HybridACD	Δ AVS	$\Delta\%$
Gemini-2.5-Flash	0.1116	0.0087	0.2612	0.0168	−0.1029	−92.2%
GPT-4o-mini	0.0307	0.0007	0.1752	0.0055	−0.0300	−97.6%
GPT-5.4-mini	0.4909	0.0107	1.0423	0.0227	−0.4802	−97.8%
MiniMax-M3	0.1266	0.0005	0.3756	0.0048	−0.1261	−99.6%
Mistral-Medium-3.5	0.0792	0.0087	0.2494	0.0168	−0.0705	−89.1%
Mistral-Small-4	0.0740	0.0023	0.2421	0.0156	−0.0717	−96.8%
Average	0.1522	0.0053	0.2910	0.0137	−0.1469	−95.5%

4.3 Forecasting Accuracy

Table 3 reports Brier Scores on 242 real-world questions. HybridACD improves the Brier Score for all five models with complete evaluation, with reductions ranging from 2.4% to 17.3%.

Table 3 Brier Score (BS, ↓) and Calibration Error (CE, ↓) on 242 real-world questions. HybridACD improves BS for all models, confirming the absence of Goodhart collapse. † marks cases where the increase in Calibration Error remains under investigation.

Model	Brier Score		Calibration Error		Δ BS	Goodhart?
	Baseline	HybridACD	Baseline	HybridACD		
Gemini-2.5-Flash	0.141	0.130	0.195	0.185	−7.8%	No
GPT-4o-mini	0.205	0.200	0.161	0.105	−2.4%	No
MiniMax-M3	0.123	0.116	0.161	0.117	−5.7%	No
Mistral-Medium-3.5	0.202	0.167	0.090	0.138†	−17.3%	No
Mistral-Small-4	0.211	0.202	0.124	0.127†	−4.3%	No

These results show that HybridACD does not suffer from the Goodhart effect. The reason is that the system does not directly optimize model weights with respect to a consistency metric; instead, it only constrains the final output probability according to mathematical rules. However, Calibration Error increases for the two Mistral models, suggesting that hard constraints may compress the model’s confidence distribution toward constraint boundaries. This limitation should be addressed in future work through softer penalty mechanisms.

4.4 Comparison with Baselines

Table 4 places HybridACD next to common forecasting baselines. Compared with ArbitrageForecaster, HybridACD achieves substantially lower AVS while requiring much lower cost per question. Notably, HybridACD + MiniMax-M3 obtains a Brier

Score of 0.116, outperforming the more expensive CoT configurations shown in the comparison.

Table 4 Comprehensive comparison of forecasting methods on the Scraped benchmark. HybridACD achieves the best AVS with negligible additional cost and no Goodhart risk. Costs are estimated using API prices from June 2026.

Method	AVS ($\downarrow$)	Brier Score ($\downarrow$)	Cost/Question (USD)	Goodhart
BasicForecaster (GPT-4o-mini)	~ 0.412	0.205–0.231	\$0.002	—
CoT-Forecaster (GPT-4o)	~ 0.287	~ 0.198	\$0.018	—
CoT-Forecaster (o1-preview, best)	~ 0.089	~ 0.170	\$0.05+	—
ArbitrageForecaster ($N = 4$)	~ 0.161	~ 0.245	$\sim \$2,500$	Overfit
HybridACD + GPT-4o-mini (ours)	0.0007	0.200	\$0.019	No
HybridACD + Gemini-2.5-Flash (ours)	0.0087	0.130	$\sim \$0.07$	No
HybridACD + MiniMax-M3 (ours)	0.0005	0.116	$\sim \$0.08$	No
HybridACD + Mistral-Medium-3.5 (ours)	0.0087	0.167	\$0.115	No

Three observations can be drawn from the table. First, HybridACD improves consistency more strongly than post-hoc calibration methods while avoiding many rounds of optimization. Second, forecasting accuracy is not sacrificed for consistency. Third, the additional cost of HybridACD mainly comes from adversarial rewriting and probability extraction, making it more suitable for practical deployment than multi-round iterative methods.

4.5 Ablation Study

Table 5 evaluates the role of each component in HybridACD. When TCD is removed, AVS increases from 0.0007 to 0.0147, indicating that TCD is the main component responsible for hard consistency guarantees. When the adversarial agent is removed, AVS increases to 0.0012 and the Brier Score slightly worsens, showing that adversarial rewriting improves robustness against linguistic artifacts.

Table 5 Ablation study on GPT-4o-mini. Full HybridACD achieves the best consistency, while each module contributes independently.

Configuration	AVS	Brier Score	Hard Guarantee?
BasicForecaster	0.0307	0.205	No
HybridACD ^{−adv} (TCD only)	0.0012	0.202	Yes
HybridACD ^{−tcd} (Adversarial only)	0.0147	0.206	No
Full HybridACD	0.0007	0.200	Yes

5 Discussion

The experimental results show that HybridACD achieves a balance among three objectives: probability consistency, forecasting accuracy, and inference cost. Methodologically, the key difference of HybridACD lies in the location of intervention. Instead of modifying probabilities after the model has answered or updating weights during training, the system intervenes at the final probability extraction step. As a result, the model can still reason freely, but its numerical output must lie within the valid domain.

The adversarial component tests the model’s robustness to meaning-preserving linguistic transformations. This is particularly important for rules such as NEGATION and PARAPHRASE, where models are easily affected by surface lexical cues. TCD, in contrast, acts as a mathematical guardrail that ensures the final probability does not violate bounds derived from consistency rules.

Nevertheless, HybridACD has several limitations. First, hard constraints may increase Calibration Error for some models, especially when the interval $[\ell, u]$ is too narrow compared with the model’s natural confidence distribution. Second, sequential processing reduces the ability to parallelize large consistency sets. Third, discretizing probability values with a step size of 0.01 is suitable for many practical forecasting tasks, but it may be insufficiently fine-grained for applications requiring higher numerical precision.

6 Conclusion

This report presented HybridACD, a hybrid architecture for improving the probability consistency of LLM forecasters through adversarial rewriting and token-constrained decoding. Building on prior work on consistency checks, probability-consistent optimization, self-consistency, and constrained decoding, HybridACD chooses decoding time as the intervention point in order to avoid two major limitations of previous approaches: the cost of post-hoc calibration and the Goodhart risk of directly optimizing the model.

Experiments show that HybridACD reduces average AVS by 95.5% and improves Brier Score on the models with complete evaluation. This suggests that probability consistency does not necessarily have to be traded off against forecasting accuracy if constraints are applied at the right location. Future work can extend the system in three directions: using soft logit bias instead of hard blocking, improving the handling of deeply nested conditional chains, and evaluating HybridACD on LLM forecasters that integrate RAG or real-time data.

Acknowledgements

References

- [1] Paleka, D., Sudhir, A.P., Alvarez, A., Bhat, V., Shen, A., Wang, E., Tramèr, F.: Consistency Checks for Language Model Forecasters (2025). <https://arxiv.org/abs/2412.18544>

- [2] Lowin, J.: An intuitive guide to how llms work. Medium / Personal Blog (2024)
- [3] Love, B.C.: Backwards compatible: The strange math behind word order in ai. Cognitive Science Blog (2026)
- [4] Anonymous: Probability consistency in large language models: Theoretical foundations meet empirical discrepancies. Submitted to Transactions on Machine Learning Research (Rejected) (2025)
- [5] Yang, Y., Ren, H., Lu, Z., Wang, K., Shi, W., Zhou, A., Pan, J., Zhan, M., Li, H.: Probability-consistent preference optimization for enhanced LLM reasoning. In: Findings of the Association for Computational Linguistics: ACL 2025, pp. 6435–6448 (2025). <https://aclanthology.org/2025.findings-acl.333/>
- [6] Zhou, Z., Yuhao, T., Li, Z., Yao, Y., Guo, L.-Z., Ma, X., Li, Y.-F.: Bridging Internal Probability and Self-Consistency for Effective and Efficient LLM Reasoning (2025). <https://arxiv.org/abs/2502.00511>
- [7] Koo, T., Liu, F., He, L.: Automata-based constraints for language model decoding. In: First Conference on Language Modeling (2024). <https://openreview.net/forum?id=BDBdblmyzY>
- [8] Yao, J.-M., Chen, H.-Y., Tang, Z.-X., Tan, B.-J., Peng, S.-W., Xie, B.-C., Su, S.-F.: Token Constraint Decoding Improves Robustness on Question Answering for Large Language Models (2025). <https://arxiv.org/abs/2506.09408>
- [9] Huang, J.Y., Sengupta, S., Bonadiman, D., Lai, Y.-A., Gupta, A., Pappas, N., Mansour, S., Kirchhoff, K., Roth, D.: Deal: Decoding-time alignment for large language models. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics, pp. 26280–26300 (2025). <https://doi.org/10.18653/v1/2025.acl-long.1274>
- [10] Schall, M., Melo, G.: The hidden cost of structure: How constrained decoding affects language model performance. In: Proceedings of the 15th International Conference on Recent Advances in Natural Language Processing, pp. 1074–1084 (2025). <https://aclanthology.org/2025.ranlp-1.124/>
- [11] Mousavi, J., Termehchy, A.: Towards consistent language models using controlled prompting and decoding. arXiv preprint (2024)